Multi-Agent AI Workflow Engine

Simulating an Autonomous Software Development Team

PROGRESS REPORT · END OF WEEK 8 OF 12

Marco Perozo · Gabriel Marte · Lewdy Aquino · Alvin Almonte
Department of Computer Science · Southwest Minnesota State University

Repository

github.com/marcoaperozo/Capstone-MAWE

Phases 1–3 complete · engine v1.0 · 255 automated tests passing · evaluation phase next

Where the project stands

Three of the four phases are complete and on schedule. The engine is feature-complete for the research question; what remains is measuring it against a real model.

Phase	Weeks	Deliverable	Status
1 · Setup & prototyping	1–2	Repository, CI, database schema, pipeline on a state graph (M1)	COMPLETE
2 · Role implementation	3–5	Product Manager, Developer and Code Reviewer agents (M2)	COMPLETE
3 · Memory & state	6–8	Loop hardening, context compression, integration suite, web UI (M3)	COMPLETE
4 · Testing & evaluation	9–12	Benchmark suite, single-model baseline, final report (M4)	NEXT

M1 – M3 all met

The three-role pipeline runs end to end, provably terminates, stays inside a token budget, and every run is fully traceable.

Engine v1.0

255 automated tests, green on Python 3.11, 3.12 and 3.13. No API key needed and no cost per run.

What is left

Phase 4: a 20-task benchmark with hidden tests, a single-model baseline, and the cost-per-success comparison.

What was delivered in Weeks 5 to 8

Commit history

One commit per week, each carrying that week's milestone result in its message.

Week	Deliverable	What it means in practice	Tests
5	Code Reviewer agent (M2)	The specification now carries executable checks. The Reviewer runs them in the sandbox and the measurement overrides what the model claims.	145
6	Loop-prevention hardening	Routing and reporting unified into one policy function; two new detectors (oscillation, repeated issues); 100,000-run stress sweep.	212
7	Context compression	Latest iteration kept in full, older ones collapsed into a progress log under an explicit token budget.	233
8	Integration, web UI, v1.0 (M3)	Whole-stack integration suite, optional FastAPI front end, and a 30-point check that every layer still works together.	255

Next: Weeks 9–12 build the benchmark suite and run the comparison against a single model.

Making the Reviewer more than a second opinion

An acceptance criterion written in English can simply be claimed to pass by a model that never ran anything. So the Product Manager also emits executable checks — an invocation and the output it must produce — and the Reviewer runs them in the sandbox before forming a verdict.

Situation	What the engine does
Model approves, but a check failed	Overridden to REJECT. Escalated to critical on a crash or timeout.
Model rejects, all checks pass, every criterion covered	Downgraded to MINOR, so a reviewer objecting to naming cannot stall the loop.
Model rejects, all checks pass, some criteria uncovered	Severity STANDS — a criterion nobody tested may genuinely be unmet.
Model output unparseable	Verdict comes from the execution evidence alone.

Demonstrated, not asserted

Scripted to always approve, the Reviewer still rejected code whose rounding was wrong — a defect only visible by running it. The Developer fixed it and approval required all three checks to pass.

A flaw this surfaced

The downgrade originally fired whenever all checks passed, even when most criteria had none. That quietly reintroduced false approval. It now requires full criterion coverage.

Making the loop stop — five mechanisms

All of the policy lives in one pure function of the run state, so every branch is testable without calling a model — and a randomised stress sweep can drive 100,000 sequences through it in about a second.

1 · Hard iteration cap

Configurable ceiling, default five. The backstop that always holds.

2 · Severity threshold

Only critical and major block approval, so trivia cannot stall the loop.

3 · No-progress detection

Two consecutive identical file hashes mean the Developer is not responding to feedback.

4 · Oscillation detection (new)

A hash seen earlier, not just immediately before. Catches A → B → A cycling, invisible to mechanism 3.

5 · Repeated-issue detection (new)

A normalised fingerprint of the complaints. Renumbering or rewording the same objection is not progress.

Fallback routing

Exiting without approval never fails the run: the best version is returned with a report of which mechanism fired.

Stress result: 100,000 randomised adversarial sequences across four policy configurations. 0 failed to terminate, 0 exceeded their cap. With the cap set to 20, the detectors bounded the worst run to 8.

Keeping long runs affordable

The Developer prompt replayed every previous file in full, so context grew linearly while the budget did not move. Older iterations now collapse into a structured progress log.

Context size as a run gets longer (4,000-token budget)

Iterations	2	5	10	20	40	80
Uncompressed (tokens)	1,293	3,260	6,539	13,104	26,234	52,494
Compressed (tokens)	688	787	954	1,291	1,966	3,316
Reduction	46.8%	75.9%	85.4%	90.1%	92.5%	93.7%

Structured, not generative

A model-written summary costs tokens, is non-deterministic, and can silently drop the detail that mattered. The fields a later iteration needs are known in advance, so they are recorded exactly and for free. The generative alternative is implemented behind the same interface for the Phase 4 comparison.

H5 ablation (condition C′)

End to end, with and without compression: 2,080 tokens against 513, a 75.3% reduction, with an identical termination reason and iteration count. H5 targets 40%. Confirming 'no quality drop' against a real model is Phase 4 work.

What is working — and what is not yet

Stated plainly so the remaining risk is visible rather than discovered later.

Component	State	Evidence or what remains
Orchestration — LangGraph state graph	**Working**	Runs end to end; whole-stack integration suite
Loop prevention — five mechanisms	**Working**	100,000 randomised runs, none exceeded its cap
Product Manager agent	**Working**	Valid specs on 5/5 sample prompts, both languages
Developer agent	**Working**	Complete multi-file output; placeholder content rejected
Code Reviewer agent	**Working**	Verdict grounded in sandbox execution, not self-report
Execution sandbox	**Working**	Timeout, crash and path escape all handled
Context compression	**Working**	93.7% reduction at 80 iterations, always within budget
Persistence — 11 tables	**Working**	Every run replayable and costable
Web UI (FastAPI)	**Working**	Optional; the engine and CLI do not depend on it
Real-model adapters (Anthropic, OpenAI)	**Untested**	Written behind the same interface; no live API call made yet — needs a key, a config change rather than a code change
Benchmark suite and baselines	**Not started**	Weeks 9–12, as scheduled

How to verify any of this

Clone the repository and run these. No API key, no network, no cost.

`pytest -q`	255 tests, about ten seconds.
`python -m mawe.cli "Build a snake game" -v`	Watch the pipeline run live.
`python scripts/week5_m2_check.py`	The Reviewer rejects a defect it was scripted to approve.
`python scripts/week6_loop_stress.py`	100,000 adversarial sequences; none fail to terminate.
`python scripts/week7_memory_check.py`	Context growth with compression on and off.
`python scripts/week8_m3_check.py`	30 checks across every layer.
`uvicorn mawe.web.app:app`	The web front end, if FastAPI is installed.

Repository: github.com/marcoaperozo/Capstone-MAWE

Setup: pip install -r requirements.txt && pip install -e . · full walkthrough in docs/DEMO-GUIDE.md